ECE408/CS483/CSE408 Fall 2024

Applied Parallel Programming

Lecture 15
Parallel Computation Patterns – Parallel Scan (Prefix Sum) – Part 2

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Course Reminders

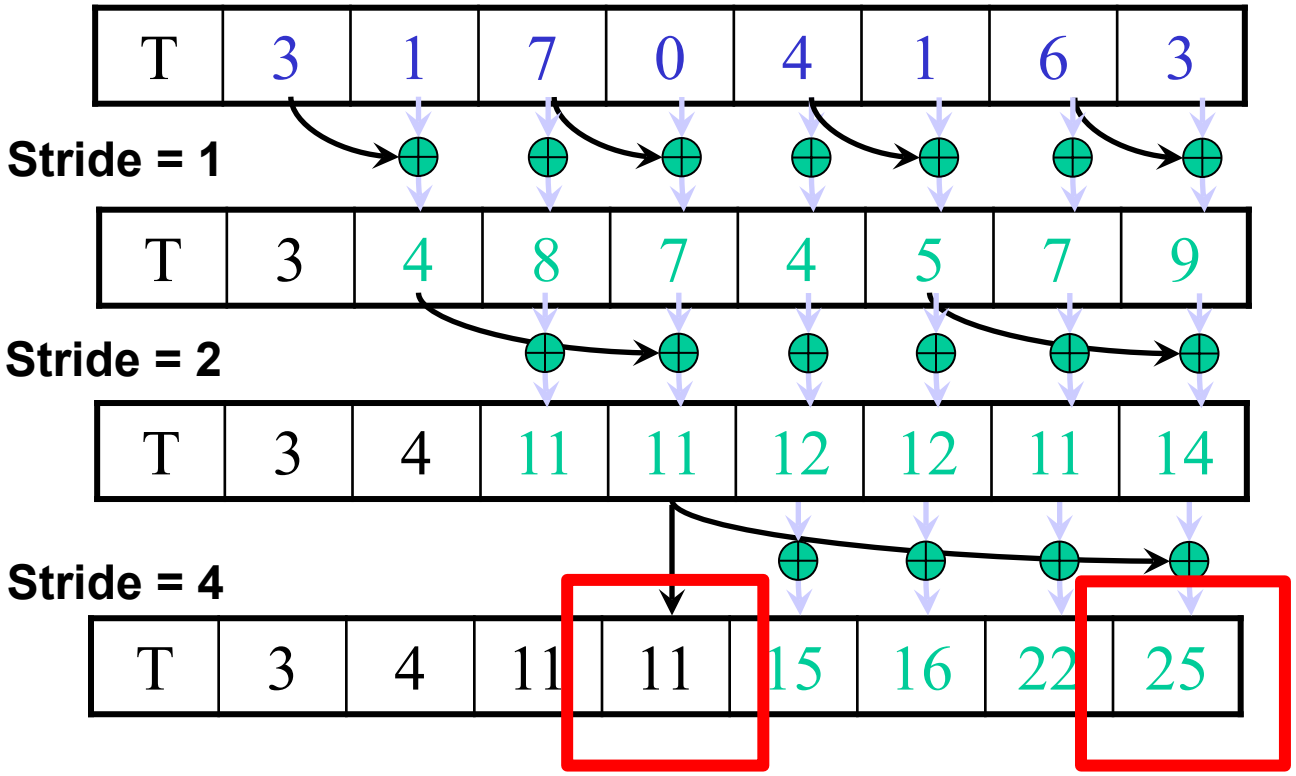
- Lab 6
  - Due 10/25/24
  - Implement one or more kernels and their associated host code to perform parallel scan on a 1D list. The scan operator used will be addition. You should implement the work-efficient kernel discussed in today's lecture. Your kernel should be able to handle input lists of arbitrary length. However, for simplicity, you can assume that the input list will be at most 2,048 * 2,048 elements.
- Project Milestone 2: Matrix multiplication-based kernel
  - Due 11/08/24
- MT1
  - Regrade requests are due Thursday, October 24, by 8pm

Objective

- To master parallel scan (prefix sum) algorithms
  - Work-efficiency vs. latency
  - Brent-Kung Tree Algorithm
  - Hierarchical algorithms

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

A Kogge-Stone Parallel Scan Algorithm



© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Work Efficiency Analysis

- A Kogge-Stone scan kernel executes log(n) parallel iterations
  - The steps do (n-1), (n-2), (n-4),..(n- n/2) add operations each
  - Total # of add operations: n * log(n) - (n-1) → O(n*log(n)) work

- This scan algorithm is not very work efficient
  - Sequential scan algorithm does *n* adds
  - A factor of log(n) hurts: 20x for 1,000,000 elements!
  - Typically used within each block, where n ≤ 1,024

- A parallel algorithm can be slow when execution resources are saturated due to low work efficiency

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Improving Efficiency

- A common parallel algorithm pattern:

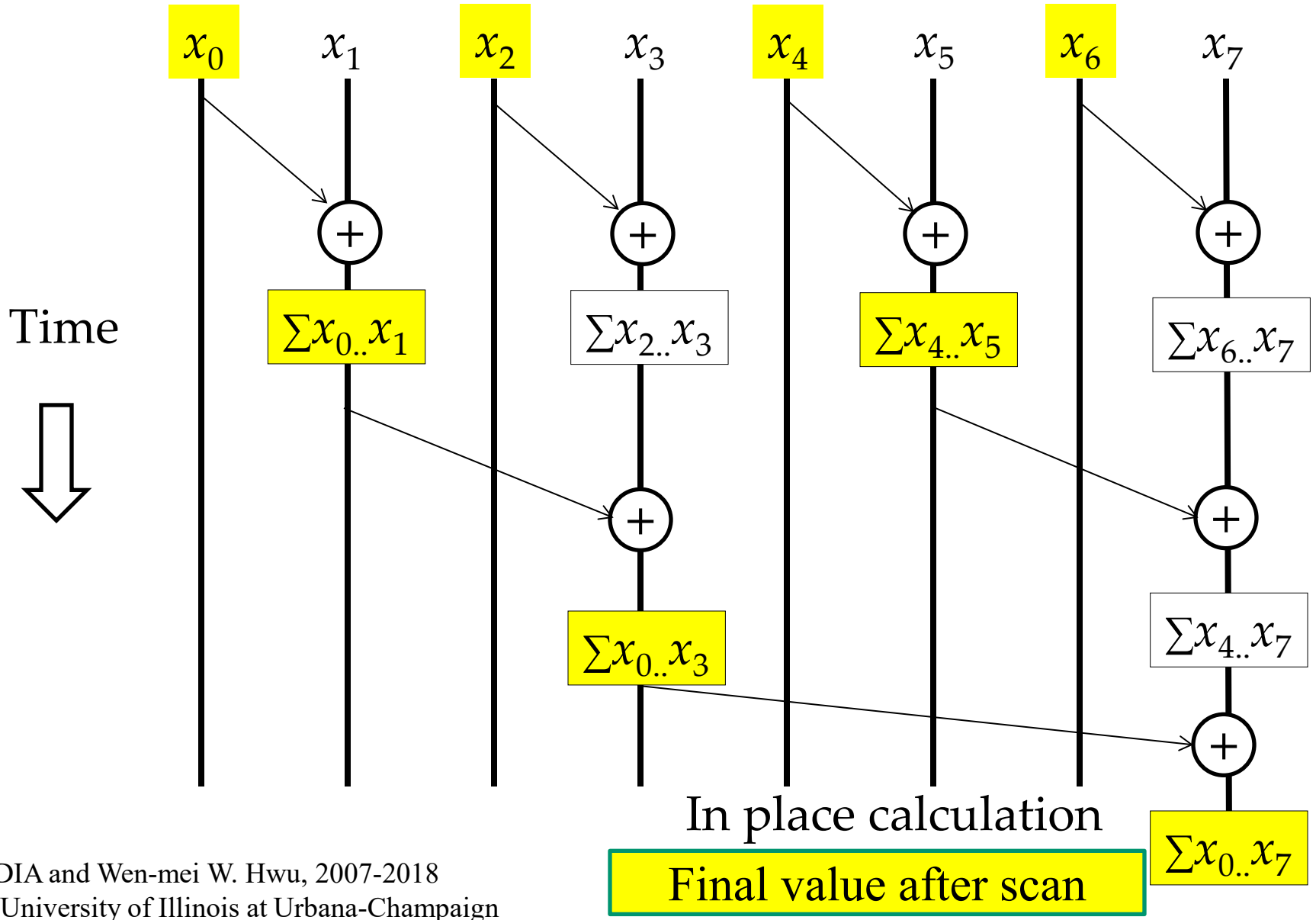
  Balanced Trees

  - Build a balanced binary tree on the input data and sweep it to and from the root
  - Tree is not an actual data structure, but a conceptual pattern

- For scan:
  - Traverse down from leaves to root building partial sums at internal nodes in the tree
    - Root holds sum of all leaves
  - Traverse back up the tree building the scan from the partial sums

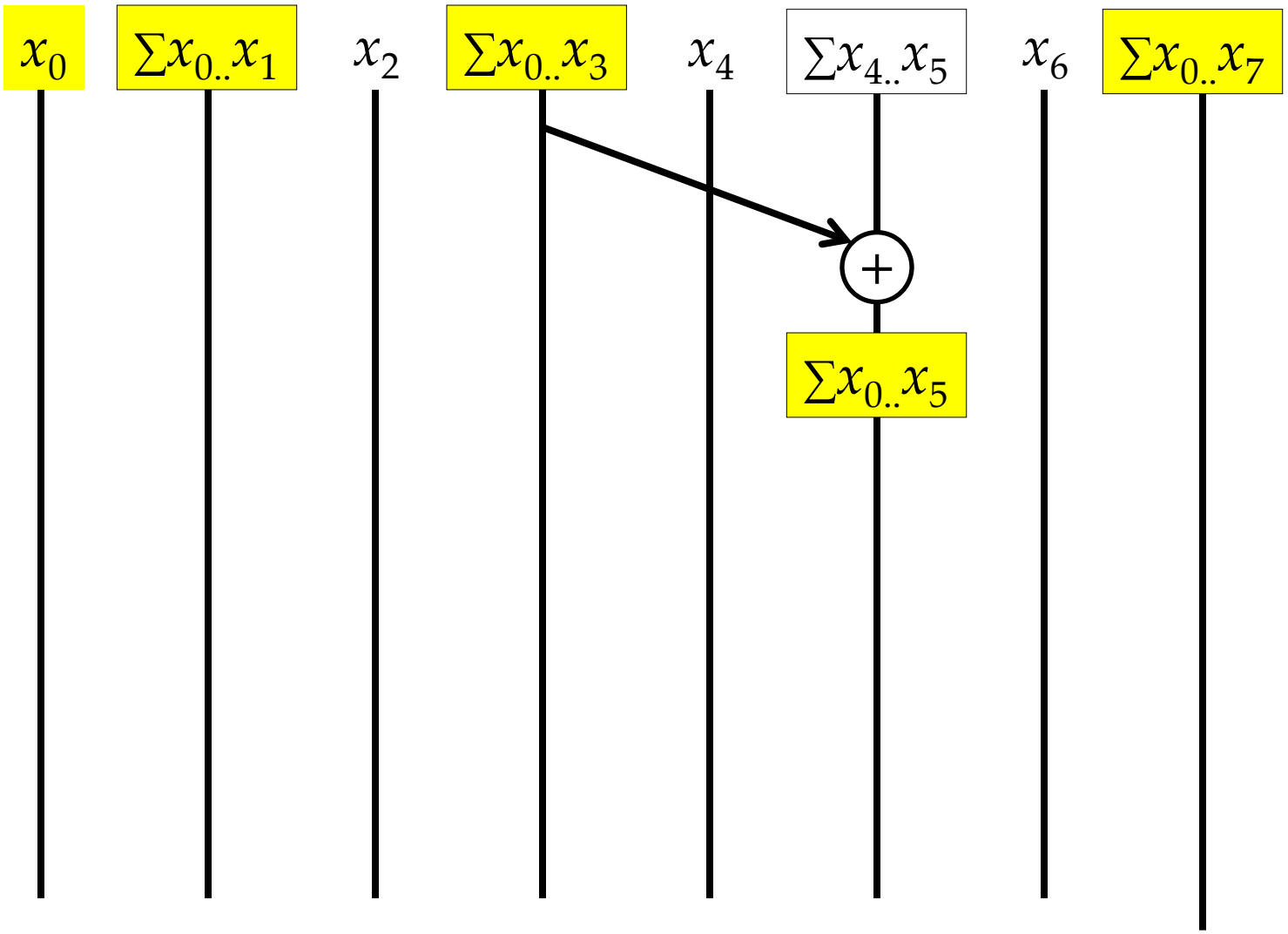
© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Brent-Kung Parallel Scan Step



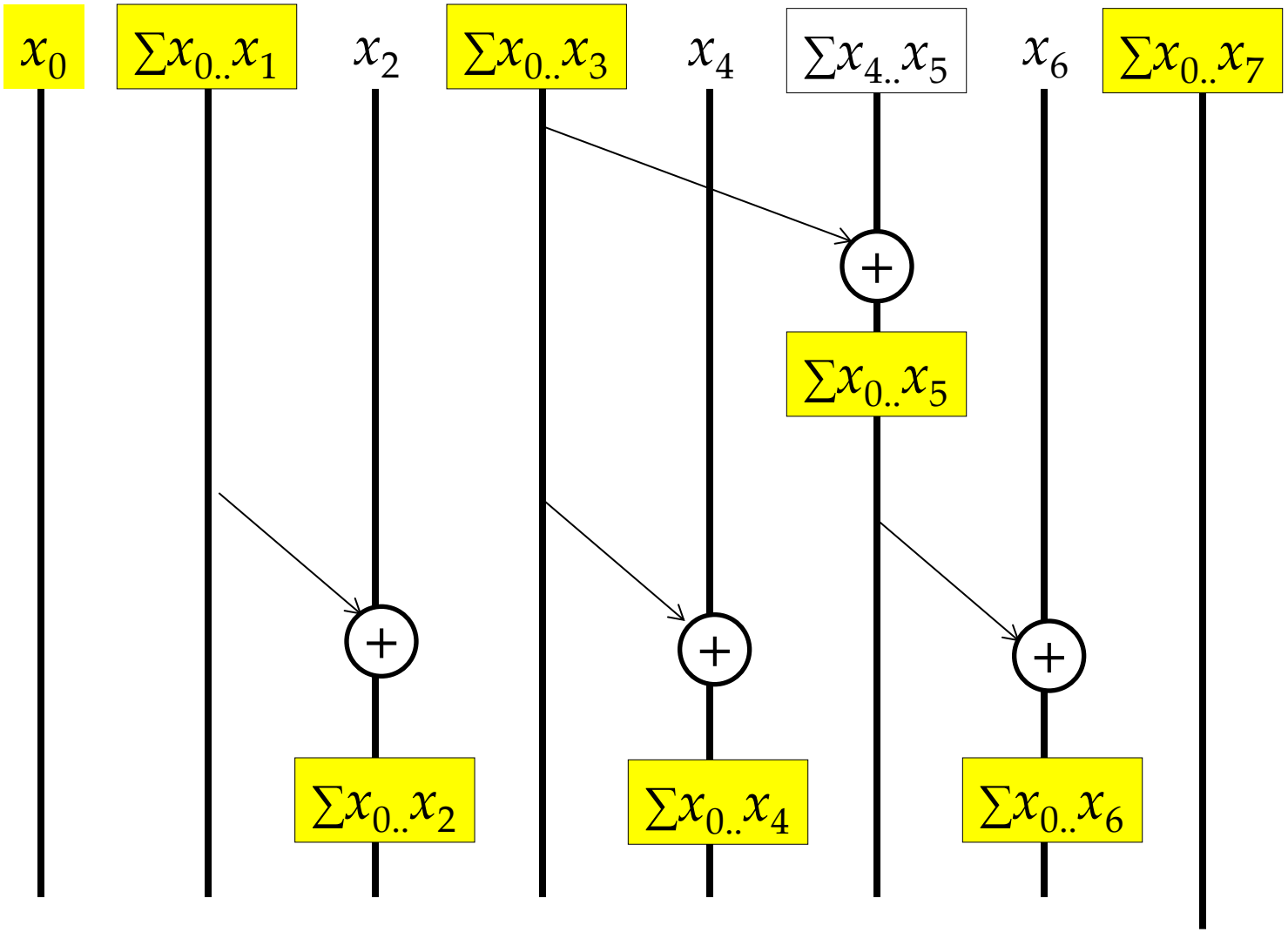
© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Inclusive Post-Scan Step



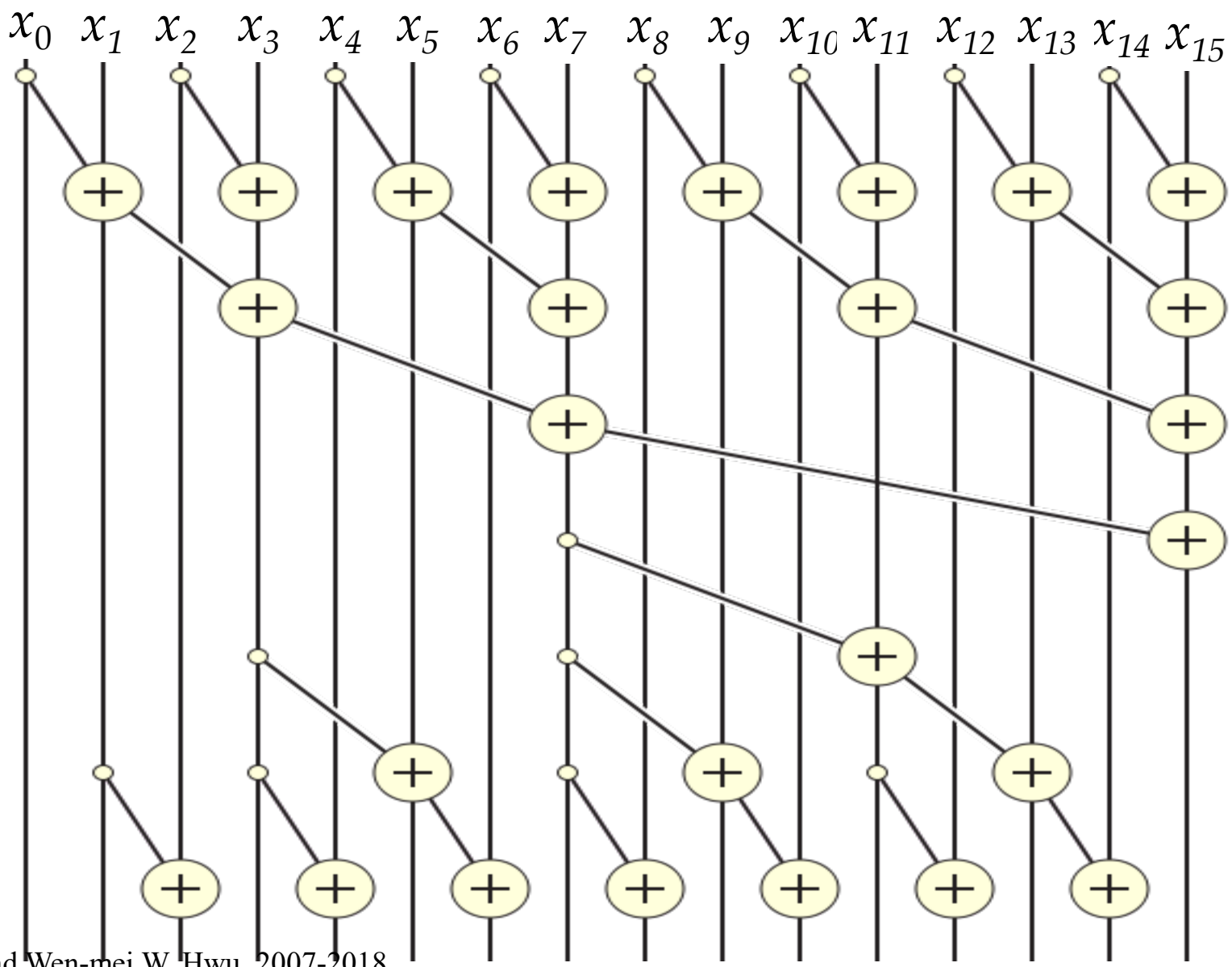
© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Inclusive Post Scan Step



© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Putting it Together (Data View)



© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Reduction Step Kernel Code

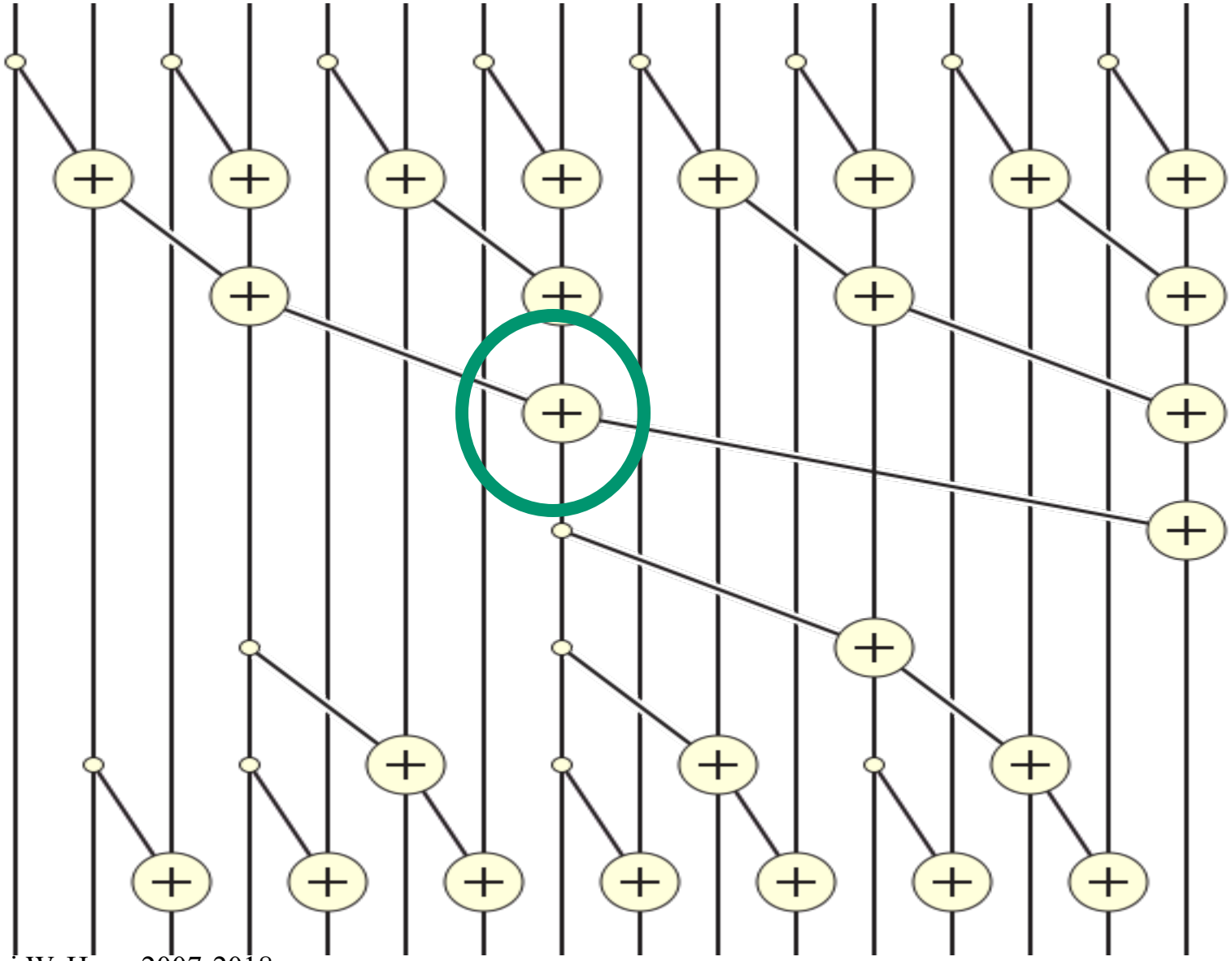
```
// float T[2*BLOCK_SIZE] is in shared memory
// for previous slide, BLOCK_SIZE is 8

int stride = 1;
while(stride < 2*BLOCK_SIZE) {
    __syncthreads();
    int index = (threadIdx.x+1)*stride*2 - 1;
    if(index < 2*BLOCK_SIZE && (index-stride) >= 0)
        T[index] += T[index-stride];
    stride = stride*2;
}
```

```
// In our example,
// threadIdx.x+1    = 1, 2, 3, 4, 5, 6, 7, 8
// stride = 1, index = 1, 3, 5, 7, 9, 11, 13, 15
```


© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Putting it Together



© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Post Scan Step (Distribution Tree)

```
int stride = BLOCK_SIZE/2;
while(stride > 0) {
    __syncthreads();
    int index = (threadIdx.x+1)*stride*2 - 1;
    if ((index+stride) < 2*BLOCK_SIZE)
        T[index+stride] += T[index];
    stride = stride / 2;
 }
```

```
// In our example,
// BLOCK_SIZE=8 stride=4, 2, 1
// for first iteration, active thread = 0 index = 7, stride = 11
```

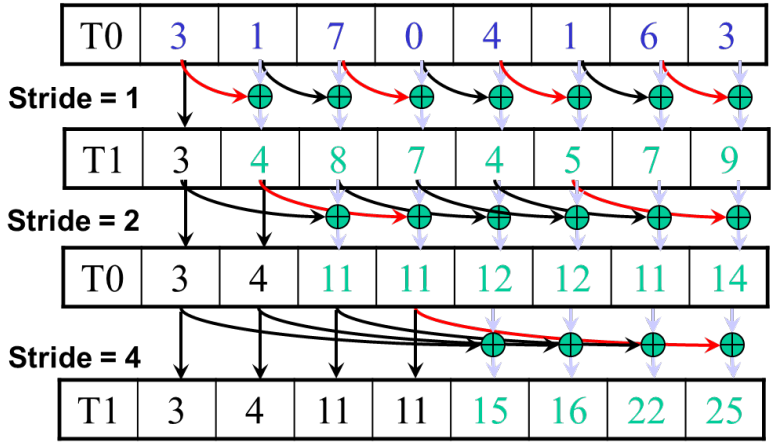

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

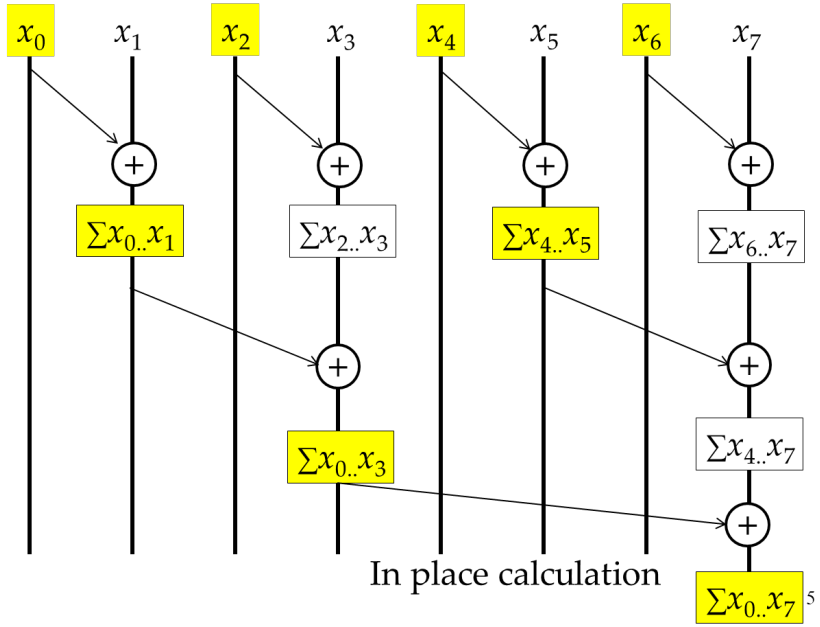
Work Analysis

- The parallel Scan executes 2*log(n) parallel iterations
  - log(n) in reduction and log(n) in post scan
  - The iterations do n/2, n/4,..1, (2-1), …., (n/4-1), (n/2-1) useful adds
  - In our example, n = 16, the number of useful adds is 16/2 + 16/4 + 16/8 + 16/16 + (16/8-1) + (16/4-1) + (16/2-1)
  - Total adds: (n-1) + (n-2) – (log(n) -1) = 2*(n-1) – log(n)→ O(n) work
- The total number of adds is no more than twice of that done in the efficient sequential algorithm
  - The benefit of parallelism can easily overcome the 2× work when there is sufficient hardware

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Kogge-Stone vs. Brent-Kung



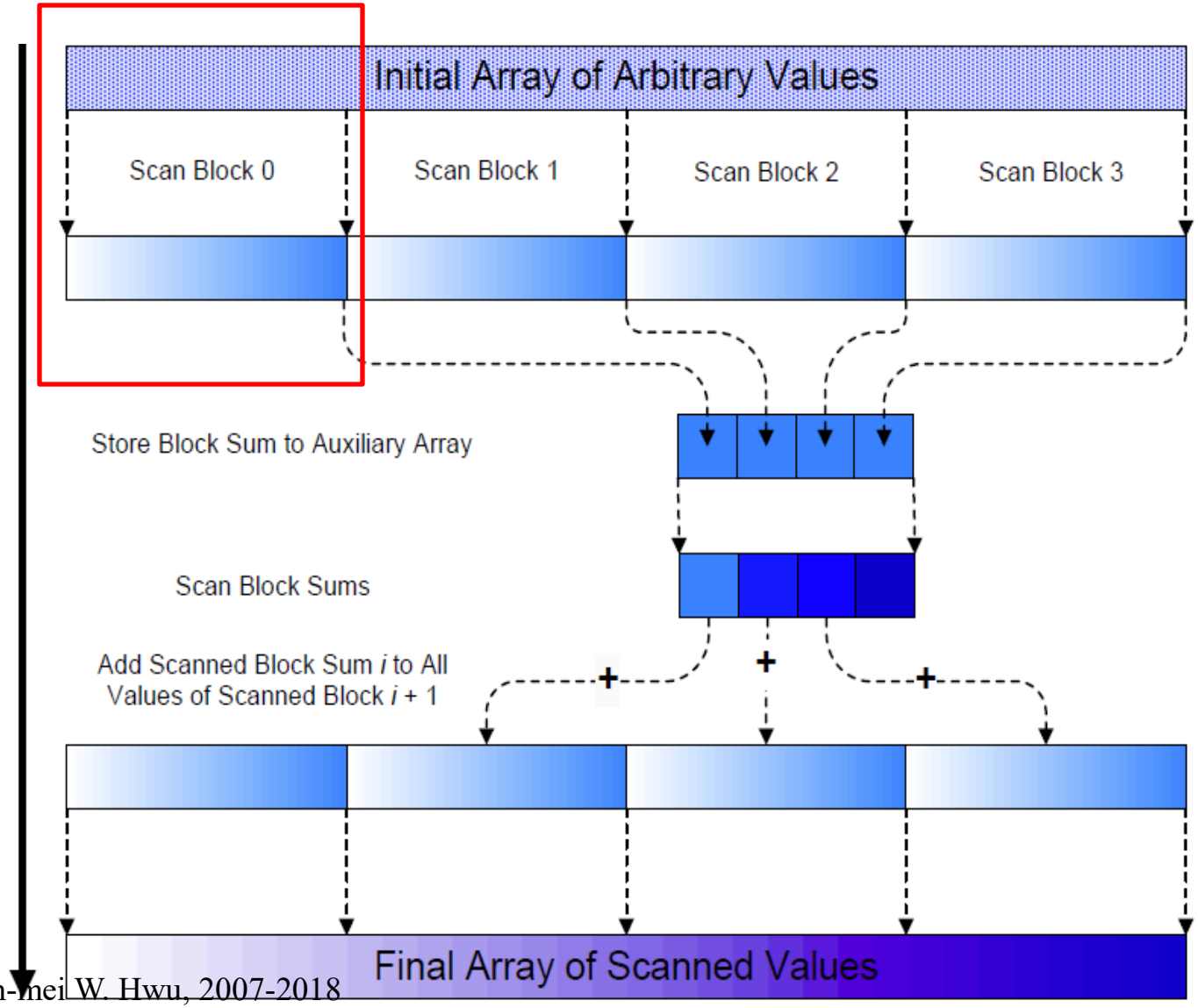


- Brent-Kung uses half the number of threads compared to Kogge-Stone
  - Each thread should load two elements into the shared memory
- Brent-Kung takes twice the number of steps compared to Kogge-Stone
  - Kogge-Stone is more popular for parallel scan with blocks in GPUs

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Overall Flow of Complete Scan
A Hierarchical Approach

Scan kernel



© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Using Global Memory Contents in CUDA

- Data in registers and shared memory of one thread block are not visible to other blocks
- To make data visible, the data has to be written into global memory
- However, any data written to the global memory are not visible until a memory fence. This is typically done by terminating the kernel execution
- Launch another kernel to continue the execution. The global memory writes done by the terminated kernels are visible to all thread blocks.

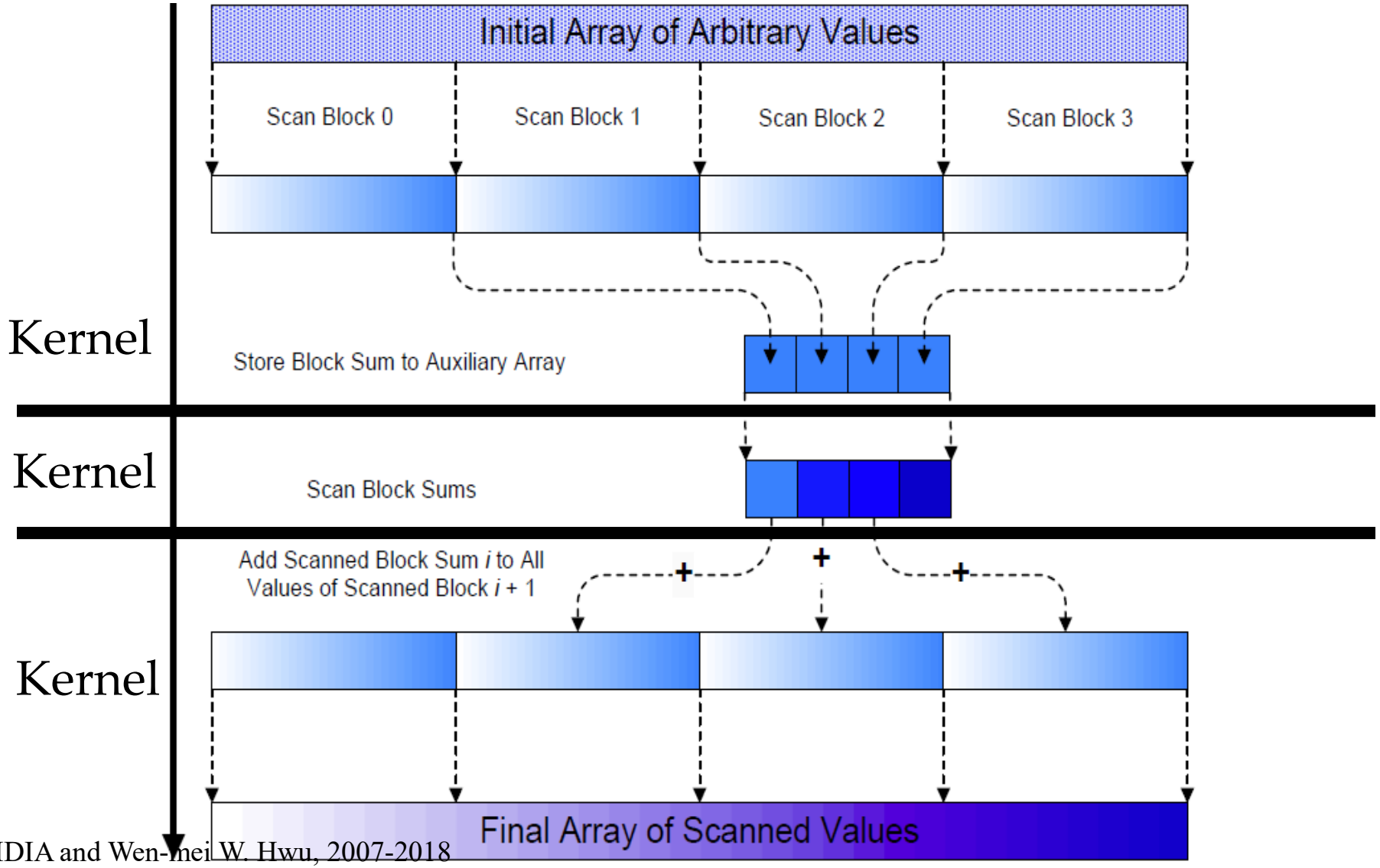
© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Scan of Arbitrary Length Input

- Build on the scan kernel that handles up to 2*blockDim.x elements from Brent-Kung
  - For Kogge-Stone, have each section of blockDim.x elements assigned to a block
- Have each block write the sum of its section into a Sum array using its blockIdx.x as index
- Run parallel scan on the Sum array
  - May need to break down Sum into multiple sections if it is too big for a block
- Add the scanned Sum array values to the elements of corresponding sections

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Overall Flow of Complete Scan
A Hierarchical Approach



© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

(Exclusive) Scan Definition

Definition: *The exclusive* scan *operation takes a binary associative operator* $\oplus$*, and an array of n elements*

$$[x_0, x_1, \ldots, x_{n-1}]$$

and returns the array

$$[0, x_0, (x_0 \oplus x_1), \ldots, (x_0 \oplus x_1 \oplus \ldots \oplus x_{n-2})].$$

Example: If $\oplus$ is addition, then the exclusive scan operation on

$$[3\ \ 1\ \ 7\ \ 0\ \ 4\ \ 1\ \ 6\ \ 3],$$

would return $[0\ \ 3\ \ 4\ \ 11\ \ 11\ \ 15\ \ 16\ \ 22]$.

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Why Exclusive Scan

- To find the beginning address of allocated buffers

- Inclusive and Exclusive scans can be easily derived from each other; it is a matter of convenience

	[3 1 7 0 4 1 6 3]
Exclusive	[0 3 4 11 11 15 16 22]
Inclusive	[3 4 11 11 15 16 22 25]

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

A simple exclusive scan kernel

- Adapt an inclusive, Kogge-Stone scan kernel
  - Block 0:
    - Thread 0 loads 0 into (shared) XY[0]
    - Other threads load (global) X[threadIdx.x-1] into XY[threadIdx.x]
  - All other blocks:
    - All thread load X[blockIdx.x*blockDim.x+threadIdx.x-1] into XY[threadIdex.x]

- Similar adaption for Brent-Kung kernel but pay attention that each thread loads two elements
  - Only one zero should be loaded
  - All elements should be shifted by only one position

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

ANY MORE QUESTIONS?
READ CHAPTER 8

© David Kirk/NVIDIA and Wen-mei W. Hwu, 2007-2018
ECE408/CS483/ University of Illinois at Urbana-Champaign

Problem Solving

- Q: Suppose we have a kernel function that performs partial sum reduction. The block dimension is (64,1,1). During the **second** iteration of the for loop, is there any warp that has a control divergence?

```
__shared__ int partialSum[2*BLOCK_SIZE];
for (int stride = blockDimx.x; stride >= 1; stride = stride/2)  {
   __syncthreads();
   if (threadIdx.x < stride) partialSum[t] += partialSum[t + stride];
}
```

- A:
  - No

© Volodymyr Kindratenko, 2023 ECE408/CS483,
University of Illinois, Urbana-Champaign

Problem Solving

- Q: Consider a kernel that performs Brent-Kung scan algorithm and assume that there are 1024 elements in a section, and a warp size is 32. In which iteration will there be at least one warp that has a control divergence? The stride is 1 for the first iteration.

- A:
  - In 6th iteration

© Volodymyr Kindratenko, 2023 ECE408/CS483,
University of Illinois, Urbana-Champaign

Problem Solving

- Q: Suppose we use Brent-Kung in a hierarchical approach to perform parallel scan on a 1D input array of 2^{42} elements. We use 1024 threads per block in all our Brent-Kung kernels and our GPU supports at most 2048 blocks per grid. Each block processes 2048 elements. What is the best approximation of the number of floating-point add operations performed per thread block in the reduction and post scan steps in the Brent-Kung kernel (summed together)?

- A:
  - 2048 x 2

© Volodymyr Kindratenko, 2023 ECE408/CS483,
University of Illinois, Urbana-Champaign